

Débuter Avec C#

Base du langage C#

C# est un langage introduit par Microsoft en 2000. C'est un langage objet avec un typage statique fort, une syntaxe héritée du C/C++ et une philosophie très proche de Java. C# est un langage phare du framework .net qui se popularise pour la conception de sites Web (ASP), d'ERP (Sharepoint), de scripting et d'applications lourdes.

A l'origine considéré comme une pâle copie de Java, le C# a ensuite bénéficié d'une politique de développement très dynamique par rapport à Java, resté en désérence plusieurs années par Sun avant son rachat par Oracle. Aujourd'hui, C# est un langage de programmation moderne avec une bibliothèque standard très riche et des outils de développement avancés.

La syntaxe du C# est très proche du C et de Java et ne devrait donc pas poser de difficulté si vous avez déjà pratiqué l'un de ces langages. Les éléments de base de la structuration du code sont les accolades `{ }`, qui définissent les blocs, et le point virgule `;`, qui marque la fin d'une instruction. Le saut de ligne est un caractère d'espacement comme un autre (une instruction peut s'étaler sur plusieurs lignes).

Les plus courageux peuvent consulter le [guide officiel de programmation C#](#) et les [spécifications complètes du langage](#).

Les commentaires

Les **commentaires** sont introduits comme en C/C++ et Java :

- sur une seule ligne: tout ce qui suit les caractères `//` est ignoré jusqu'à la fin de la ligne;
- sur plusieurs lignes: tout ce qui se trouve entre les balises `/*` et `*/` est ignoré.

```
// Je suis un commentaire sur une seule ligne.
```

```
/* Je suis un commentaire sur plusieurs lignes.  
   J'apparais grisé dans les éditeurs offrant une coloration syntaxique */
```

Les variables

Un programme stocke l'information dans des **variables**. Une variable correspond à un espace de la mémoire auquel on donne un nom, on parle d'**identificateur**, et auquel on associe un **type de donné**. Le contenu de cet espace mémoire peut alors être manipulé (lu ou écrit) en le désignant par son identificateur.

Déclaration

Avant de pouvoir utiliser une variable il faut la **déclarer** avec la syntaxe suivante :

Syntaxe

```
TypeDeDonnee Identificateur;  
TypeDeDonnee Identificateur = ValeurInitiale;
```

Exemples :

```
int unEntier;  
float unNombreFloatant = 3.0; // déclaration de la variable unNombreFloatant de type  
float et initialisation  
Bitmap uneImage; // déclaration de la variable uneImage de type Bitmap
```

Avertissement

Une variable peut être déclarée sans être **initialisée** : sa valeur est alors indéfinie. Essayer de lire une variable alors que sa valeur est indéfinie est une erreur de compilation en C#.

Un identificateur est un nom donné par l'utilisateur et il est fortement conseillé de choisir un nom explicite, c'est à dire qui décrit le rôle de la variable. La seule exception à cette règle concerne les variables de boucle pour lesquelles les identificateurs courts `i`, `j`, `k` sont réservés. Ces aspects sont traités en détail dans les [recommandations générales de nommage](#) du guide de programmation C#.

```
int a = 20; // à éviter : impossible de voir à quoi correspond cette variable à partir de  
son nom  
int largeurPixel = 20; // beaucoup mieux: on sait immédiatement que la variable correspond  
à une largeur  
// et que l'on compte en nombre de pixels
```

Les identificateurs ne peuvent pas être des [mots clés](#) du langage (`int`, `if`, `else`...). Ils sont sensibles à la casse (majuscule, minuscule) et doivent obéir à certaines règles : ne pas commencer par un chiffre, ne pas contenir certains signes... ([l'ensemble des règles](#) est plutôt complexe, mais en pratique tenez vous en aux caractères alphanumériques et tout ira bien).

Note

En C#, on suit la **convention** [Camel Case](#) : les différents mots constituant un identificateur commencent par une majuscule et ne comportent pas de caractère de séparation. De plus, pour les identificateurs de variables, le premier mot commence par une minuscule. Pour plus d'information sur le sujet.

```
int jeSuisLaNormeCamelCase;
```

Types de données

Le langage C# est

- fortement typé : il est interdit de mettre une valeur dans une variable si les types de la variable et de la valeur ne sont pas compatibles;
- manuellement typé : les informations de typage sont données par le développeur;
- statiquement typé : le typage est vérifié par le compilateur et toute erreur empêche la compilation du programme;
- dynamiquement typé : le CLR vérifie également l'intégrité des types à l'exécution.

Chaque variable est associée à un type donné. Ces types sont divisés en 2 catégories:

1. Les types **valeurs** composés des **types primitifs** (types de base du langage) et des structures (non traitée dans ce cours). Pour cette catégorie, l'emplacement mémoire correspondant à la variable contient directement la valeur concernée.

Type de données	Genre	Taille (octets)	Valeurs possibles
<code>byte</code>	nombre entier	1	00 à 255255
<code>short</code>	nombre entier	2	$-2^{15}-2^{15}$ à $2^{15}-12^{15}-1$
<code>int</code>	nombre entier	4	$-2^{31}-2^{31}$ à $2^{31}-12^{31}-1$
<code>long</code>	nombre entier	8	$-2^{63}-2^{63}$ à $2^{63}-12^{63}-1$
<code>float</code>	nombre réel	4	$\pm 1.5 \times 10^{-45} \pm 1.5 \times 10^{-45}$ à $\pm 3.4 \times 10^{38} \pm 3.4 \times 10^{38}$
<code>double</code>	nombre réel	8	$\pm 5.0 \times 10^{-324} \pm 5.0 \times 10^{-324}$ à $\pm 1.7 \times 10^{308} \pm 1.7 \times 10^{308}$
<code>decimal</code>	nombre réel	16	$-7.9 \times 10^{28} - 7.9 \times 10^{28}$ à $7.9 \times 10^{28} 7.9 \times 10^{28}$
<code>bool</code>	valeur booléenne	1	<code>true</code> ou <code>false</code>
<code>char</code>	caractère	2	<code>U+0000</code> à <code>U+FFFF</code> (caractères Unicode)

2. Les types **références** composés des **classes** définies par l'utilisateur ou présentes dans l'API (en particulier les tableaux). Dans ce cas l'emplacement mémoire correspondant à la variable contient une référence vers un objet. Nous reviendrons en détail sur le comportement de cette catégorie de types.

Constantes littérales

Les **constantes littérales**, c'est-à-dire, les valeurs rentrées directement dans le code sont typées :

```
1 // est un entier (le type exacte byte, short, int ou long sera déterminé par le
contexte)
1.0 // est un double
1.0f // est un float
'c' // est un caractère
true // est un booléen
```

Transtypage

Le **transtypage** ou **cast** désigne l'action de transformer une valeur d'un type de données vers un autre type. Pour caster une valeur dans un nouveau type, il faut indiquer le nom du nouveau type entre parenthèses devant la valeur:

```
double d = 2.1;
int i = (int)d; // d est casté en int, i contient la valeur 2: la partie décimale est supprimée
```

Si le transtypage n'est pas possible, il y aura une erreur, soit au moment de la compilation, soit au cours de l'exécution du programme.

Le C# n'est pas un langage *très* fortement typé: le compilateur accepte de réaliser automatiquement certains transtypages si ces derniers peuvent se faire sans perte. On parle de **conversion implicite**.

```
int x = 2;
double d = x; // 2 est converti en double, le compilateur ne dit rien
int y = d;    // ERREUR, la conversion automatique de double vers int n'est pas possible
```

Parmi les types primitifs, les conversions implicites suivantes sont

possibles: `double` ←← `float` ←← `long` ←← `int` ←← `short` ←← `byte` .

Attention

Les opérations de conversions implicites par le compilateur peuvent paraître annodines mais elles sont un élément clef d'un des composants les plus complexes des langages de programmation moderne: la **résolution de nom**. La [résolution de nom](#) consiste, pour le compilateur, à déterminer quel est l'identificateur correcte à utiliser en fonction du contexte.

Ce mécanisme est très complexe dans les langages modernes, chaque couche d'abstraction générant de nouvelles règles (polymorphisme paramétriques, polymorphismes d'héritage, généricité, template, duck typing). Cela mène à des résultats surprenant lorsque l'on ne maîtrise pas bien ces mécanismes. Par exemple, ruby et javascript sont connus pour leurs [conversions implicites hasardeuses](#) alors que les interactions entre conversions implicites et templates de la librairie standard du C++ génère des messages d'erreurs [interminables et difficilement compréhensibles](#) .

Expression

Une **expression** est une combinaison de variables, d'opérateurs, de constantes et d'appels de fonction. Comme les variables, les expressions ont une valeur et un type. Par contre les expressions ne sont pas identifiées par un nom. Le calcul de la valeur d'une expression est appelé **évaluation**.

```
int x = 2;
int y = x + 2; // x + 2 est une expression de type int et de valeur 4
bool b = y >= 5; // y >= 5 est une expression de type bool et de valeur false
```

Les **règles de priorité** classiques sont utilisées lors de l'évaluation d'une expression (par exemple la multiplication est prioritaire sur l'addition).

Les **parenthèses** permettent de changer l'ordre d'évaluation des différentes parties d'une expression et de clarifier la lecture.

Opérateurs simples

La [liste des opérateurs du C#](#) est assez longue, les plus communs sont:

- Opérateurs de comparaison : ==, !=, <, >, <=, >=
- Opérateurs logiques : || (ou), && (et), ! (négation)
- Opérateurs numériques: +, -, *, /, % (modulo)

Lorsque les 2 opérandes numériques d'un opérateur binaire ne sont pas du même type, l'opérande du type le plus petit est automatiquement convertie vers le type le plus grand. Par exemple:

```
1.0 + 2 // équivalent à 1.0 + 2.0: 2 est converti en double
```

Opérateur d'affectation =

L'**opérateur d'affectation** permet de modifier la valeur d'une variable (à gauche du signe égal) avec une nouvelle valeur (à droite du signe égal). On dit que les variables sont des **left-value** car elle peuvent être mise à gauche d'un signe égal: elles correspondent à une zone de mémoire modifiable.

```
int x;
x = 4;
```

Important

L'opérateur d'affectation fonctionne toujours par copie. La variable reçoit une copie de la valeur à droite du signe égal.

```
int x = 1;
int y = 0;

y = x; // y = 1
y = y + 1; // y = 2, x n'est pas modifié et vaut toujours 1
```

Dans le cas d'un type référence, la valeur stockée dans la variable étant la référence, l'opérateur d'affectation copie cette référence et non l'objet référencé.

Il existe une variante de l'opérateur d'affectation pour chaque opérateur binaire. Par exemple l'opérateur dérivé `+=` permet d'ajouter une valeur à une variable:

```
int x = 3;
x += 2; // équivalent à x = x + 2;
```

Remarque: Le type d'une expression d'affectation `x = y` est le type de `x` et sa valeur est la valeur de `y`.

Structures de contrôle

On retrouve les structures de contrôle classiques : **if**, **else**, **for**, **while**.

if

Syntaxe

```
if ( condition )
{
    // instructions à réaliser si CONDITION vaut true
} else {
    // instructions à réaliser si CONDITION vaut false
}
```

for

Syntaxe

```
for ( initialisation; condition; instructionDIteration )
{
    // bloc d'instructions à réaliser tant que condition vaut true
}
```

Workflow d'exécution d'une boucle for

while

Syntaxe

```
while ( condition )
{
    // bloc d'instructions à réaliser tant que condition vaut true
}
```

Les fonctions

Une **fonction** est une portion de code qui effectue une tâche ou un calcul. Une fonction est désignée par un identificateur qui permet de l'appeler à partir d'autres endroits du programme, c'est-à-dire de demander à exécuter la portion de code qui correspond à la fonction. La fonction peut retourner une valeur, c'est-à-dire **renvoyer un résultat**. Une fonction peut prendre des **arguments** ou **paramètres** qui désignent les données d'entrée sur lesquelles travaille la fonction.

Déclaration

La déclaration d'une fonction se fait en 2 parties :

- On commence par donner son **prototype** qui contient toutes les informations d'identification de la fonction: son nom, ses paramètres, son type de retour...
- Puis on donne son corps entre accolades qui contient les instructions de la fonction.

Syntaxe

```
TypeDeRetour IdentificateurFonction(TypeParametre1 identificateurParametre1, TypeParametre2
identificateurParametre2, ...) // prototype
{
    // début du corps
    // instructions de la fonction
    return valeurDeRetour;
} // fin du corps
```

Une fonction qui ne retourne pas de résultat utilise le type de retour special `void` (qui signifie rien). Les fonctions qui ne retournent rien sont appelées **procédures**.

Syntaxe

```
void IdentificateurFonction(TypeParametre1 identificateurParametre1, TypeParametre2
identificateurParametre2, ...)
{
    // instructions de la fonction
    return; // optionnel, remarquez l'absence de valeur après l'instruction return
}
```

Attention

Les fonctions dont le type de retour est différent de `void` doivent se terminer par l'exécution d'une instruction `return` suivie d'une valeur dont le type est compatible avec le type de retour de la fonction. Le compilateur vérifie que, quelque soient les paramètres d'entrée de la fonction, l'exécution de la fonction se termine toujours par une instruction `return`. Ainsi le code suivant est faux et provoque le message d'erreur : *Tous les chemins de code ne retournent pas nécessairement une valeur* :

```
int F3(int x)
{
    if(x<2)
    {
        return 1;
    }
}
```

Note

Par convention, les identificateurs de fonction en C# suivent la norme Camel Case mais, contrairement aux identificateurs de variable, on fera commencer leur premier mot par une majuscule.

Appel de fonction

Une fonction est appelée en indiquant son identificateur suivi, entre parenthèses, des valeurs des paramètres:

Syntaxe

```
IdentificateurFonction(valeurParamètre1, valeurParamètre2,...);
```

Un **appel de fonction** dont le type de retour est différent de `void` est une expression qui a pour type le type de retour de la fonction et pour valeur la valeur retournée par la fonction.

```
int Fonction2(int a)
{
    a += 1;
    return a;
}

void Fonction1()
{
    int b = 7;
    b = Fonction2(b); // Fonction2(b) est une expression de type int et de valeur 8
}
```

Important

Lors de l'appel d'une fonction, le passage des paramètres est toujours réalisé par copie : la fonction reçoit une copie de la valeur passée en paramètre. Si la variable est de type valeur, il s'agit d'une copie de cette valeur. Si la variable est de type référence, il s'agit d'une copie de la référence.

```
void Increment(int a)
{
    a += 1;
}

...

int x = 0;
Increment(x);
int y = x; // y vaut 0 !
```

Portée des variables

Un **bloc** est une portion de code délimitée par une paire d'accolades : `{ }`. Les blocs sont organisés de manières hiérarchiques : chaque bloc peut contenir d'autres blocs et ainsi de suite. On parle de **bloc de niveau supérieur** (le contenant) et de **bloc de niveau inférieur** (le contenu).

Les **variables** dites **locales** sont :

- les paramètres d'une fonction,
- les variables déclarées à l'intérieur d'une fonction,
- les variables déclarées dans une boucle.

Une variable locale a une **portée d'utilisation** limitée. Elle ne peut donc pas être utilisée à l'extérieur de sa portée, d'où le caractère local. Sa portée commence à sa déclaration/définition. Pour un paramètre de fonction ou une variable déclarée dans un bloc, la portée se termine à la fin de son bloc : lors de l'accolade fermante. Dans le cas d'une boucle, la variable cesse d'exister lorsque la boucle se termine.

Exemple:

```
int Fonction2(int a)
{
    a += 1;
    return a;
} // a cesse d'exister ici

void Fonction1()
{
    int b = 7;
    for(int i=0; i < 10; i++)
    {
        b = Fonction2(b);
    } // i cesse d'exister ici
} // b cesse d'exister ici
```

Les tableaux

En utilisant un **type existant**, le langage permet de créer des **tableaux**. Un tableau peut être vu comme une liste, un vecteur ou une matrice d'éléments du même type. Chaque **case** ou **cellule** d'un tableau stocke un élément du type donné.

Déclaration et création

Etant donné un type `T`, la notation `T[]` désigne le nouveau type **tableau d'éléments de type T**. Par exemple `int[]` désigne le type *tableau d'entiers* qui est différent du type *entier*. On peut déclarer des variables avec ce type de données.

Pour créer un tableau, on utilise l'opérateur `new`. Les tableaux sont de **type référence**.

Tableaux unidimensionnels:

Syntaxe

```
TypeDesElements [ ] monTableau1; // déclaration d'un tableau
TypeDesElements [ ] monTableau2 = new TypeDesElements [tailleDuTableau]; // déclaration et création d'un tableau
int [ ] monTableau3 = {1, 5, 6, 7}; // déclaration d'un tableau et initialisation explicite
```

Tableaux multidimensionnels :

Syntaxe

```
TypeDesElements [,] monTableau1; // déclaration d'un tableau bidimensionnel
TypeDesElements [,,] monTableau2; // déclaration d'un tableau tridimensionnel
TypeDesElements [,] monTableau3 = new TypeDesElements [nombreDeLignes, nombreDeColonnes] ;
// déclaration et initialisation d'un tableau
int [,] monTableau4 = { {1, 5, 6},
                       {2, 9, 7} }; // déclaration d'un tableau 2d et initialisation explicite
```

Les variables de type tableau sont comme les autres : elles doivent être initialisées avant d'être utilisées. Lors de la création d'un tableau, chaque cellule du nouveau tableau est **initialisée par défaut**. Ainsi, si vous créez un tableau d'entiers, chaque case sera mise à zéro.

Accès aux éléments d'un tableau

Les cases d'un tableau sont numérotées de 0 à la taille du tableau moins un. Le numéro d'une case est appelé son **indice** et permet d'accéder à cette case.

Syntaxe

```
int [] monTableau1 = {1, 2, 3};
monTableau1[0] = 4; //modification de la valeur de la case d'indice 0: monTableau1 = {4, 2, 3}
int a = monTableau1[2]; // lecture de la valeur de la dernière case du tableau: a = 3

int [,] monTableau2 = { {1, 2, 3},
                        {4, 5, 6}};
monTableau2[0, 0] = 7; // monTableau2 = { {7, 2, 3}, {4, 5, 6}}
int b = monTableau2[1, 2]; // b = 6
```

L'accès à un indice invalide d'un tableau (en dehors d'un tableau) déclenche une erreur à l'exécution.

Taille d'un tableau

Pour connaître la taille d'un tableau unidimensionnel on utilise la propriété `Length`:

```
int [] monTableau1 = {1, 2, 3};
int taille = monTableau1.Length; // taille = 3
```

Pour connaître la taille d'un tableau bidimensionnel on utilise la méthode `GetLength(dimension)`:

```
int [,] monTableau1 = { {1, 2, 3},
                        {4, 5, 6} };

int lignes = monTableau1.GetLength(0); // 2
int colonnes = monTableau1.GetLength(1); // 3
```

Parcours des tableaux

Avec une boucle **for** ou une boucle **foreach** (si les indices des éléments ne nous intéressent pas).

```
bool [] monTableau = {true, true, false};

for (int i = 0; i < monTableau.Length; i++)
{
    ... monTableau[i] ...
}

foreach (bool valeur in monTableau) // se lit: "pour toute valeur de type bool dans monTableau"
{
    ... valeur ...
}
```

Chaines de caractères

Les chaines de caractères sont désignées par le type `String` (ou `string` avec une minuscule) qui est de **type référence**. Les constantes de type chaine de caractères s'écrivent entre **double quotes** `"`.

```
string s1 = "Bonjour";  
String s2 = "Bonjour";  
String s3 = ""; // chaine de caractères vide
```

L'**opérateur de concaténation** `+` permet de mettre 2 chaines de caractères bout à bout. Tous les types de données sont implicitement convertibles en chaines de caractères avec l'opérateur `+`. Ainsi, une expression de la forme « chaine de caractères » + *n'importe quoi* est toujours valide et son résultat est une chaine de caractères.

```
double d = 42.0;  
bool b = true;  
  
String s1 = "La valeur de d est " + d; // s1 = "La valeur de d est 42"  
String s2 = "" + b;                  // s2 = "true"
```

Une chaine de caractères est composée de caractères au format unicode sur 2 octets (UTF16). Le backslash `\` sert de caractère d'échappement pour obtenir les caractères spéciaux : par exemple `\n` pour le retour à la ligne, `\t` pour une tabulation, `\\` pour obtenir un backslash.

Le caractère `@` placée devant une constante de type `String` désactive l'interprétation des caractères spéciaux. Cela est particulièrement pratique pour les chemin d'accès aux fichiers :

```
String filename = @"C:\dossier\fichier1.txt" ;
```

L'API .net propose de nombreuses méthodes pour [manipuler les chaînes de caractères](#) : (`Replace`, `Substring`, `ToLower`, `ToUpper`, `Insert` ...), test sur le contenu (`IsEmpty`, `Compare`, `Contains`, `Equals` ...), longueur de la chaine (`Length`), conversion vers les types de données primitifs (ex: `ToInt32`).

Attention

Les chaînes de caractères sont **immuables**: on ne peut pas modifier une chaîne après sa création. Toutes les opérations sur les chaînes de caractères entraînent la création d'une nouvelle chaîne de caractères. Par exemple l'initialisation de la variable `s` dans le code ci-dessous :

```
int i = 1;
double d = 2;
bool b = false;
```

```
String s = " i = " + i + ", d = " + d + ", b = " + b + "."; // s = " i = 1, d = 2, b = false."
```

implique un total de 13 chaînes de caractères:

- les 4 constantes entre double quote;
- les 3 chaînes issues des conversions implicites de `i`, `d` et `b`;
- les 6 chaînes créées par chacun des opérateurs `+`.